(3 Shifts) Greedy Scheduling Heuristics

October 13, 2026

1 Introduction

This document describes a constructive greedy heuristic for yearly employee shift scheduling. The problem involves assigning employees to shifts across a full calendar year, respecting a set of hard and soft constraints.
The heuristic operates in three sequential phases:

1. **Phase 1 (Minimum Coverage)**: For each day, employees are ranked by a priority score and greedily assigned to shifts until all minimum staffing requirements are met.
2. **Phase 2 (Ideal Coverage)**: After minimum requirements are satisfied, additional assignments are made to approach ideal staffing targets. This phase is presented in two independent formulations, V1 and V2 (see the note below).
3. **Phase 3 (Remaining Employees)**: A final employee-centric pass that attempts to assign any remaining available working days to employees who have not yet reached their annual maximum, subject to all feasibility constraints.

Two independent formulations of Phase 2 : This document actually describes **two complete, mutually exclusive algorithms**, not one. Both share an identical Phase 1 and an identical Phase 3; they differ only in how Phase 2 is carried out. Two independent formulations of Phase 2 are presented, **V1** and **V2**, and a single run uses exactly one of them, never both:

- **V1** (day-centric) mirrors the structure of Phase 1: it iterates over days in $\tilde{D}$ and, within each day, over employees ranked by a score function.
- **V2** (employee-centric) inverts this structure: it iterates over employees and, for each, pops the single highest-demand day still pending, attempting one assignment per employee per pass until every employee's demand queue is exhausted.

The heuristic does not use any mathematical programming solver; all decisions are made locally per day using the scoring function described in Section 6.1.

2 Indices

Index	Description
E	Set of Employees
D	Set of days, $D = \{1, 2, \ldots, \lvert D\rvert\}$, where $\lvert D\rvert = \begin{cases} 365, & \text{if common year} \\ 366, & \text{if leap year} \end{cases}$
S	Set of Shifts, $S = \{1, 2, 3\}$ (Morning, Afternoon, Night)
T	Set of Teams
T^*	Set of Teams order by number of Shifts s in decrescent order
$R \subseteq E$	Set of Employees ranked by Score
T_e	Set of teams t that employee can work
M_e	Set of days d ordered by demand per Employee e (ties are randomly order)

All Sets with $\sim$ above means that is randomly ordered (Eg. $\tilde{\text{D}}$ -Set of Random days)

3 Parameters

Parameter	Description
θ_{dst}	Minimum number of employees required on day d, shift s, team t
β_{dst}	Ideal number of employees on day d, shift s, team t
δ_e	1 if employee e is on vacation, 0 otherwise
α_{ed}	1 if employee e worked on day d, 0 otherwise
$D_{\max}$	Maximum working days per year per employee (223)
$H_{\max}$	Maximum Sundays/holidays worked per year per employee (22)
$C_{\max}$	Maximum consecutive working days (5)
W_{pace}	Weight for pace component in scoring function (0.60)
W_{seq}	Weight for streak component in scoring function (0.25)
W_{trans}	Weight for shift-transition component in scoring function (0.10)
W_{teams}	Weight for team flexibility component in scoring function (0.03)

4 State Variables

Variable	Description
TotalDays_e	Number of working days assigned to employee e so far
Streak_e	Current consecutive working-day streak for employee e
PrevShift_{ed}	Shift worked by employee e on the previous calendar day $d-1$ (or `None`)
NextShift_{ed}	Shift assigned to employee e on the next calendar day $d+1$
SunHol_e	Number of Sundays/holidays worked by employee e so far
Score_e	Priority score of employee e, updated after each day
Assignment	Dictionary mapping each employee e to a list of tuples (d, s, k)
DaysLeft_e	Remaining working days available to employee e, i.e. $D_{\max} - \text{TotalDays}_e$

5 Rules and Constraints Covered

The heuristic guarantees compliance with the following rules:

- Full coverage of minimum staffing requirements per day, shift and team (*soft constraint*);
- At most one shift assignment per employee per day (*Hard Constraint*);
- At most $C_{\max} = 5$ consecutive working days per employee (*Hard Constraint*);
- At most $D_{\max} = 223$ working days per year per employee (*Hard Constraint*);
- At most $H_{\max} = 22$ Sundays/holidays worked per year per employee (*Hard Constraint*);
- Employee vacation periods V_e must not be violated (*Hard Constraint*);
- The **ForbiddenTransitions** between assigned shifts are as follows:
  - Night → Morning
  - Night → Afternoon
  - Afternoon → Morning
- Each employee must be assigned only to allowed teams (*Hard Constraint*);

6 Auxiliary Functions of the Heuristic

6.1 Employee Scoring Function

The scoring function $\text{Score}(e, d)$ assigns a priority value to each employee e at the end of day d based on his assignments and the value is applied in next day. Higher scores indicate higher assignment priority. The score combines four components:

$$\text{Score}(e, d) = W_{\text{pace}} \cdot \phi_{\text{pace}}(e) + W_{\text{seq}} \cdot \phi_{\text{seq}}(e) + W_{\text{trans}} \cdot \phi_{\text{trans}}(e) + W_{\text{teams}} \cdot \phi_{\text{teams}}(e) \quad (1)$$

Pace component ϕ_{pace} Measures how far the employee is from the expected annual workload. Employees who have worked fewer days than expected receive a higher score:

$$\phi_{\text{pace}}(e) = \left(1 - \left(\frac{\text{Totals_Days}_e}{D_{\max}}\right)\right)^2 \quad (2)$$

Streak component ϕ_{seq} Penalises employees who are close to the maximum consecutive-day limit, favouring those with shorter current streaks:

$$\phi_{\text{seq}}(e) = \left(1 - \left(\frac{\text{Streak}_e}{C_{\max}}\right)\right)^2 \quad (3)$$

Transition component ϕ_{trans} Rewards employees whose previous shift creates a favourable transition context for the next assignment. Employees who worked a Night shift (shift 3) or an Afternoon shift (shift 2) the previous day receive a higher value, since those shifts are more constrained in what can follow them:

$$\phi_{\text{trans}}(e) = \begin{cases} 0.75 & \text{if PrevShift}_{ed} = 3 \\ 0.5 & \text{if PrevShift}_{ed} = 2 \\ 0.0 & \text{otherwise} \end{cases} \quad (4)$$

Team flexibility component ϕ_{teams} Gives a small bonus to employees who are qualified in fewer teams, so that employees with rare qualifications are assigned first:

$$\phi_{\text{teams}}(e) = \left(1 - \frac{|T_e|}{|T|}\right) \quad (5)$$

6.2 Global Assignment Order

At the start of each day, employees are ranked in decreasing order of $\text{Score}(e, d)$. Ties are broken by random shuffling (performed before the sort), ensuring fairness and diversity across heuristic runs.

6.3 Shift-Transition Validation

Before assigning shift s to employee e on day d, the heuristic checks that the transition from the previously worked shift ($s' = \text{PrevShift}_{ed}$) is valid. A transition (s, s') is valid if and only if $(s, s') \notin \text{ForbiddenTransitions}$. This enforces a minimum rest interval between consecutive work days.

In Phase 2 and Phase 3, transition validation is applied *bidirectionally*: a candidate shift s on day d must be simultaneously compatible with the shift already assigned on day $d-1$ (backward check) and with the shift already assigned on day $d+1$, if any (forward check). Formally, s is admissible if and only if:

$$(\text{PrevShift}_{ed}, s) \notin \text{ForbiddenTransitions} \quad \text{and} \quad (s,\ \text{NextShift}_{ed}) \notin \text{ForbiddenTransitions} \tag{6}$$

6.4 Consecutive Days Check

Before any assignment in Phase 2 or Phase 3, the heuristic verifies that adding employee e on day d would not create a consecutive streak exceeding $C_{\max}$. It counts the backward streak $b(e, d)$ (consecutive days worked immediately before d) and the forward streak $a(e, d)$ (consecutive days already assigned immediately after d):

$$\text{TotalStreak}(e, d) = b(e, d) + 1 + a(e, d) \tag{7}$$

The assignment is only allowed if $\text{TotalStreak}(e, d) \leq C_{\max}$. This two-sided check prevents hidden violations that arise when a new assignment fills a gap between two existing runs.

7 Heuristic Pseudocode

7.1 Phase 1 — Minimum Coverage

Algorithm 1 Phase 1 — Minimum Coverage

for each $d \in D$ **do**
  Build mandatory slot table M_d from θ_{dst}
  for each $e \in R$ **do**
    Verify Hard Constraints
    assigned $\leftarrow$ `false`
    for each $t \in T^*$ and Remaining Shifts Count SC_t **do**
      if $SC_t <= 0$ **then continue**
      end if
      for each remaining slot shift s in $M_d[t]$ **do**
        if (PrevShift$_e$, s) $\notin$ ForbiddenTransitions **then**
          Assign (d, s, t) to e; Remove s_c from $M_d[t]$
          assigned $\leftarrow$ `true`; **break**
        end if
      end for
      if assigned **then break**
      end if
    end for
    Calculate $Score_e$
  end for
end for

7.2 Phase 2 — Ideal Coverage V1

Algorithm 2 Phase 2 — Ideal Coverage V1

for each $d \in \tilde{D}$ **do**
  Build optional slot table O_d from β_{dst}
  for each $e \in R$ **do**
    if $e \in \alpha_{ed}$ **then skip**
    end if
    Verify Hard Constraints
    assigned $\leftarrow$ `false`
    for each $t \in T^*$ and Remaining Shifts Count SC_t **do**
      if $SC_t <= 0$ **then continue**
      end if
      for each slot shift s in $O_d[t]$ **do**
        if Valid Transitions (PrevShift and NextShift) **then**
          Assign (d, s, t) to e; Remove s_c from $O_d[t]$
          assigned $\leftarrow$ `true`; **break**
        end if
      end for
      if assigned **then break**
      end if
    end for
    Calculate $Score_e$
  end for
end for

7.3 Phase 2 — Ideal Coverage V2

Algorithm 3 Phase 2 — Ideal Coverage V2

while $\exists\ e \in E$ such that $M_e \neq \emptyset$ **do**
  for each $e \in E$ **do**
    $d \leftarrow M_e$.Pop() ▷ Highest-demand day for this employee
    Verify Hard Constraints
    Build optional slot table O_d
    Find teams $t \in T_e$ with optional slots in O_d, sorted by slot count
    assigned $\leftarrow$ `false`
    for each candidate team t with $|O_d[t]| > 0$ **do**
      if assigned **then break**
      end if
      for each candidate shift $s_c \in O_d[t]$ **do**
        if $(\text{PrevShift}_e,\ s_c) \in$ ForbiddenTransitions **then continue**
        end if
        if $(s_c,\ \text{NextShift}_e) \in$ ForbiddenTransitions **then continue**
        end if
        Assign (d, s_c, t) to e; Remove s_c from $O_d[t]$
        assigned $\leftarrow$ `true`; **break**
      end for
    end for
  end for
end while

7.4 Phase 3 — Remaining Employees

Phase 3 is employee-centric rather than day-centric. Its purpose is not to meet any staffing requirement, but to maximise the total number of days worked across all employees, by filling in days that were left unassigned in Phases 1 and 2.

A key aspect of this phase is the *bidirectional shift selection*: The procedure first collects all shifts that are individually valid with the previous-day assignment and all shifts valid with the next-day assignment, then takes their intersection. Only shifts present in both sets are eligible for assignment. This guarantees that a new assignment never creates a transition violation in either direction.

Algorithm 4 Phase 3 — Remaining Employees

for each $e \in E$ and $DaysLeft_e$ **do**
  if $\text{DaysLeft}_e \leq 0$ **then skip**
  end if
  counter $\leftarrow 0$
  for each $d \in \tilde{D}$ **do**
    if counter $\geq \text{DaysLeft}_e$ **then break**
    end if
    Verify hard constraints
    $S^-(e,d) \leftarrow \{s \in S : (\text{PrevShift}_{ed}, s) \notin \text{ForbiddenTransitions}\}$
    $S^+(e,d) \leftarrow \{s \in S : (s, \text{NextShift}_{ed}) \notin \text{ForbiddenTransitions}\}$
    $S^*(e,d) \leftarrow S^-(e,d) \cap S^+(e,d)$
    if $S^*(e,d) = \emptyset$ **then skip**
    end if
    Let $s^* \leftarrow \min(S^*(e,d))$, $t^* \leftarrow T_e[0]$
    Assign (d, s^*, t^*) to e;
    counter += 1
  end for
end for

Notations on shift selection in Phase 3.
S represents the shifts
S^+ represents the shifts available accordingly to the next day transition
S^- represents the shifts available accordingly to the previous day transition
S^* represents the intersection of S^+ and S^-